

Astro 142

Discussion 15

2025-11-20
Dyson Lewis

Discussion Activity

Discussion 15 Activity - multiprocessing

Download the data and the script from Bruinlearn. Follow the instructions and attach screenshots to the next slides. Upload a pdf when you are done.

```

15 def process_order(ordernumber,
16                    # Get process info
17                    proc = current_process()
18                    proc_name = proc.name
19                    proc_pid = proc.pid
20
21 # Display which core is processing this order
22 print(f"[{proc_name} - PID {proc_pid}] Processing Order {ordernumber}")
23
24 fname = basename + str(ordernumber) + '.npy'
25 data = np.load(fname)
26 logging.info(f"[{proc_name}] Loaded file: {fname}")
27
28 for i in range(data.shape[0]):
29     med = np.nanmedian(data[i])
30     if not np.isnan(med) and med != 0:
31         data[i] /= med
32
33 medspec = np.nanmedian(data, axis=0)
34
35 # Identify invalid wavelength bins (all NaN)
36 valid_cols = ~np.isnan(medspec)
37 n_bad = np.sum(~valid_cols)
38
39 if n_bad > 0:
40     logging.info(f"[{proc_name}] Order {ordernumber}: removed {n_bad} invalid all-NaN columns")
41
42 # Remove invalid columns from data and medspec
43 data = data[:, valid_cols]
44 medspec = medspec[valid_cols]
45 for i in range(data.shape[0]):
46     data[i] /= medspec
47
48 flat = data.flatten()
49 threshold = 6 * median_abs_deviation(flat, nan_policy='omit')
50
51 if threshold == 0 or np.isnan(threshold):
52     threshold = 0.1 # fallback safety valve
53
54 data[np.abs(data - 1) > threshold] = np.nan
55 if plot:
56     plt.imshow(data, aspect=15, vmin=0.95, vmax=1.05)
57     plt.title(f"Cleaned Order {ordernumber}")
58     plt.show()
59
60 print(f"[{proc_name} - PID {proc_pid}] ✓ Completed Order {ordernumber}")
61 logging.info(f"[{proc_name}] Done with {fname}")
62 return ordernumber, data

```

```

63 if __name__ == '__main__':
64     logging.basicConfig(filename='logfile.log', level=logging.INFO,
65                         format='%(asctime)s %(levelname)s: %(message)s')
66
67 # Resolve program directory and data directory (robust for interactive sessions)
68 try:
69     program_dir = os.path.dirname(os.path.abspath(__file__))
70 except NameError:
71     program_dir = os.getcwd()
72 data_dir = os.path.join(program_dir, 'discussion15_activity_data')
73
74 if not os.path.isdir(data_dir):
75     logging.error(f'Data directory not found: {data_dir}')
76     raise SystemExit(f'Data directory not found: {data_dir}')
77
78 # Change working dir to data_dir so np.load can open relative filenames easily
79 os.chdir(data_dir)
80 logging.info(f'Processing .npy files in {data_dir}')
81
82 # Default basename used by your function
83 BASE = 'WASP33 nov21_rereduced_fluxes_coadded_order_'
84
85 # Find .npy files
86 all_npy = [f for f in os.listdir('.') if f.endswith('.npy')]
87 if not all_npy:
88     logging.error('No .npy files found in data directory')
89     raise SystemExit('No .npy files found in data directory')
90
91 # Prefer files that start with the expected BASE: otherwise attempt to infer order numbers
92 matching = [f for f in all_npy if f.startswith(BASE)]
93 orders = []
94 if matching:
95     for f in matching:
96         m = re.search(r'(\d+)\.npy$', f)
97         if m:
98             orders.append(int(m.group(1)))
99 else:
100     # fallback: try to find digits before .npy and use unique numbers
101     for f in all_npy:
102         m = re.search(r'(\d+)\.npy$', f)
103         if m:
104             orders.append(int(m.group(1)))
105 # if still nothing, try to parse any trailing number after last underscore
106 if not orders:
107     for f in all_npy:
108         parts = os.path.splitext(f)[0].split('_')
109         if parts and parts[-1].isdigit():

```

```

124 # Use multiprocessing Pool to process orders in parallel
125 nprocs = min(cpu_count(), len(orders))
126 print(f'\n{"*"*60}')
127 print(f'PARALLEL PROCESSING SUMMARY')
128 print(f'{"*"*60}')
129 print(f'Total CPU cores available: {cpu_count()}')
130 print(f'Workers to use: {nprocs}')
131 print(f'Orders to process: {len(orders)} -> {orders}')
132 print(f'{"*"*60}\n')
133
134 logging.info(f'Using {nprocs} parallel workers')
135 with Pool(processes=nprocs) as pool:
136     # We run process_order with the expected BASE; process_order expects files in current cwd
137     worker = partial(process_order, basename=BASE, plot=False)
138     results = pool.map(worker, orders)
139
140     print(f'\n{"*"*60}')
141     print(f'All processing complete!')
142     print(f'{"*"*60}\n')
143
144     # results is a list of (order, data)
145     processed = {f'order_{order}': data for order, data in results}
146
147     # Save to compressed npz in program_dir (so outputs are next to the script)
148     outpath = os.path.join(program_dir, 'processed_orders.npz')
149     # np.savez_compressed accepts keyword args, so expand a dict
150     np.savez_compressed(outpath, **processed)
151     logging.info(f'Saved processed arrays to {outpath}')
152
153     # Make a simple plot: median spectrum per order (wavelength index on x-axis)
154     medians = []
155     order_labels = []
156     for order in sorted(orders):
157         arr = processed[f'order_{order}']
158         # compute median spectrum along time axis
159         med = np.nanmedian(arr, axis=0)
160         medians.append(med)
161         order_labels.append(str(order))
162
163     plt.figure(figsize=(10, 6))
164     for med, lbl in zip(medians, order_labels):
165         x = np.arange(med.size)
166         plt.plot(x, med, label=f'order {lbl}', linewidth=0.8)
167     plt.xlabel('Pixel / Wavelength index')
168     plt.ylabel('Median (normalized)')
169     plt.title('Median spectrum per order (processed)')
170     plt.legend(fontsize='small', ncol=2)
171     figpath = os.path.join(program_dir, 'median_spectra.png')

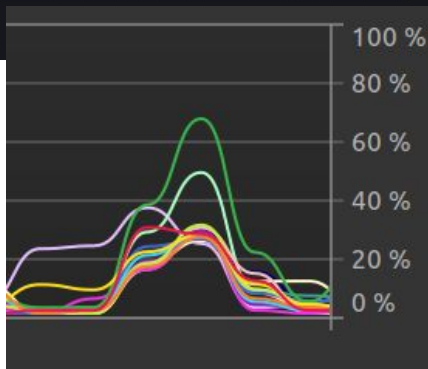
```

```
=====
Total CPU cores available: 16
Workers to use: 9
Orders to process: 9 -> [0, 1, 2, 3, 4, 5, 6, 7, 8]
=====
```

```
[ForkPoolWorker-1 - PID 23152] Processing Order 0
[ForkPoolWorker-2 - PID 23153] Processing Order 1
[ForkPoolWorker-3 - PID 23154] Processing Order 2
[ForkPoolWorker-4 - PID 23155] Processing Order 3
[ForkPoolWorker-5 - PID 23156] Processing Order 4
[ForkPoolWorker-6 - PID 23157] Processing Order 5
[ForkPoolWorker-7 - PID 23158] Processing Order 6
[ForkPoolWorker-8 - PID 23159] Processing Order 7
[ForkPoolWorker-9 - PID 23160] Processing Order 8
[ForkPoolWorker-6 - PID 23157] ✓ Completed Order 5
[ForkPoolWorker-8 - PID 23159] ✓ Completed Order 7
[ForkPoolWorker-3 - PID 23154] ✓ Completed Order 2
[ForkPoolWorker-2 - PID 23153] ✓ Completed Order 1
[ForkPoolWorker-5 - PID 23156] ✓ Completed Order 4
[ForkPoolWorker-4 - PID 23155] ✓ Completed Order 3
[ForkPoolWorker-1 - PID 23152] ✓ Completed Order 0
[ForkPoolWorker-9 - PID 23160] ✓ Completed Order 8
[ForkPoolWorker-7 - PID 23158] ✓ Completed Order 6
```

```
=====
All processing complete!
=====
```

Done.



Median spectrum per order (processed)

